

array

2024年9月21日

21:38

For an array of size N , the indices run from 0, 1, 2, . . . , $N - 1$.

- C++ compiler does not automatically check that an array index is **out of bound**.
- That is, for an array of size N , the compiler won't check if it is subscripted with an index between 0 and $N - 1$, neither at **compile-time** nor **run-time**.
- There is no compilation error for the following codes:

```
int x[10]; x[-2] = 5; x[100] = 9;
```

- When the codes are run, `x[-2] = 5;` will put the value 5 to the memory space which is 2×4 bytes (size of 2 int) **before** the array `x`. Similarly, `x[100] = 9;` will put the value 9 to the memory space which is 90×4 bytes **beyond** the array.
- This is a common cause of the **run-time error** called **segmentation fault** — your program trespasses into memory locations that may not belong to it.

define&initialize: `<data-type> <array-name> [<size>] = {<value0>...}`

If there are fewer values than the array size, the unspecified values will be zeros. It is a compilation error if there are more values than the array size. Once defined, you cannot assign values to an array using the initialization syntax.

an array variable is always passed by value.

A 2D array can be initialized in 2 ways: row by row, or like a 1D array since the array cells are actually stored linearly in the memory.

Examples

```
/* Initialize row by row */
int point[5][2] = { // An int array with 5 rows and 2 columns
    {1, 1},
    {2, 4},
    {3, 9},
    {4, 16},
    {5, 25}
};

/*
 * Initialize using the fact that the cells of a 2D
 * array actually are stored linearly in the memory
 */
int point[5][2] = { 1,1, 2,4, 3,9, 4,16, 5,25 };
```

In row-major order, the **last** dimension index runs **fastest**, while the **first** dimension index runs **slowest**.

** when calling an array **only the first size** can be not defined:

func name(array array1[][]) is false!

func name(array array1[][2]) is true and the input cannot be array[][x] when x!=2
elements are stored linearly row by row, smallest row first!!

so access column by column is less efficient

size of array=elements*sizeof(element)

elements:size1*size2*size3...->size1 may be defined or **be inferred**

```
char s2[2] = {'h', 'k'}; // no null char
char s3[3] = {'h', 'k'}; // Yes, cstring!
// s3[2] is 0, 0 is the ascii of null char
// s3[2] is null char
```

```
// same as
char s1[6] = "hk"; // null char automatically added
// we must store the null char (\0)
// we can store at most 5 other character in s1

char s2[3] = {'c', 's', 'e'}; // no null char
//char s3[3] = {'h', 'k'}; // Yes, cstring!
// s3[2] is 0, 0 is the ascii of null char
// s3[2] is null char

cout << s1 << endl;
cout << s2 << endl;
/*
// more flexible
for (int j=0; s1[j]!='\0'; ++j)
    cout << s1[j];
cout << endl;
*/
return 0;
```



no null character->out of bound access, print s2 results in printing s2+garbage value

when passing an array, it's **passing the address**->can directly modify elements

if print array in func, the result will not be array elements but array address!

判断结束: char[n]!=NULL_CHAR(NULL_CHAR is pre-defined as "\0")

#include <cstring> then can use **strlen(str)/strcmp**

str==str2 is always 0->comparing addresses not elements

strcmp(str,str2)->相同时返回0->不能直接作为bool使用

cin skip all the white spaces->只读到第一个词就停了

cin.getline(predefined_array_name, size_num, stop_sign)

- Thus, **cin >>** is not good at reading **multiple** words or even a paragraph including possibly the newline. Instead, use:

```
cin.getline(char s[], int max-num-char, char terminator);
```

- **cin.getline()** will stop when **either** (**max-num-char - 1**) characters are read, OR, the terminating character *terminator* is seen. The terminating character is removed from the input stream but is **not** read into the string.
- The C-string terminating null character is automatically inserted at the end of the read string.

cin.clear() //clear the failbit if max #chars are read